

Architectural Specification - RISC-V Core (v1)

2026-09-01

Architectural Specification - RISC-V Core (v1, RTL snapshot 2026-09-01)

1. Document purpose

Define the initial (simple) specifications for the first version of a performance-oriented RISC-V processor, to be used as a baseline for RTL implementation, verification, and synthesis.

2. First version goals

- 32-bit core for rapid ASIC flow bring-up.
- Main focus: performance compatible with limited complexity.
- Synthesizable and easily extensible core datapath.
- Bring-up integration with cache, bus, and behavioral RAM; the sparse RAM model and some cache constructs are not ASIC synthesis targets in the current state.
- No multicore support in v1.

3. ISA profile

- Base architecture: RV32I.
- Target privilege mode: M-mode only. Privileged state is not yet implemented: no CSRs, trap entry/return, or interrupts are present.
- Extensions in v1:
 - M (multiply/divide): not included in the first RTL iteration.
 - C (compressed): not included.
 - A/F/D: not included.
- Endianness: little-endian.
- Instruction alignment: 32 bit.

4. Microarchitecture (simple v1)

- 3-stage in-order pipeline:
 - IF: instruction fetch
 - ID/EX: decode + execute
 - WB: write-back to register file
- No out-of-order execution.
- No branch prediction in v1.
- Two reduced AHB-Lite master ports in the core:

- I-side read-only for instruction cache refill.
- D-side read/write for data cache refill and write-back.
- In the `riscv_top` wrapper, the two ports are arbitrated toward a unified RAM with a single outstanding transaction and fixed D-side over I-side priority.
- Control-flow handling implemented in the current RTL:
 - branch/jump resolved in ID/EX with `pc_redirect`.
 - IF/ID flush on taken redirect to eliminate the wrong-path instruction.
- Trap/exception handling not yet implemented (beyond `illegal instruction`).
- Register file:
 - 32 registers x 32 bit (x0 hardwired to 0)
 - 2 read ports, 1 write port.
- Reset vector hardcoded to `32'h0000_0000` in the IF stage.

5. Functional units

- Integer ALU:
 - add/sub
 - logic operations (and/or/xor)
 - shift (sll/srl/sra)
 - signed/unsigned comparisons (slt/sltu)
- Branch/jump implemented in the current RTL:
 - beq, bne, blt, bge, bltu, bgeu
 - jal, jalr
 - auipc (PC-relative writeback)
- Nominal load/store decode and datapath present in the current RTL:
 - lb, lh, lw, lbu, lhu
 - sb, sh, sw
- End-to-end correctness of data accesses is subject to the D-cache limitations listed in 6.2 and 10.1.
- Ecalls/ebreak/CSR/trap: not implemented in the current RTL.

5.1 Assembly instructions and current RTL status

Actual implementation status in the current RTL:

- Implemented: `add`, `sub`, `sll`, `slt`, `sltu`, `xor`, `srl`, `sra`, `or`, `and`, `addi`, `slti`, `sltiu`, `xori`, `ori`, `andi`, `slli`, `srli`, `srai`, `lui`, `auipc`, `jal`, `jalr`, `beq`, `bne`, `blt`, `bge`, `bltu`, `bgeu`, `lb`, `lh`, `lw`, `lbu`, `lhu`, `sb`, `sh`, `sw`.
- Not yet implemented: `ecall`, `ebreak`, CSR (`csrrw/csrrs/csrrc/csrrwi/csrrsi/csrrci`), `fence`, `fence.i`, and other non-base-RV32I extensions of the project.

Register-register arithmetic and logic (R-type)

Instruction	Description	opcode	funct3	funct7	Bitfield position
<code>add rd, rs1, rs2</code>	Adds <code>rs1 + rs2</code> and writes the result to <code>rd</code> .	0110011	000	0000000	<code>funct7[31:25]</code>
<code>sub rd, rs1, rs2</code>	Subtracts <code>rs2</code> from <code>rs1</code> and writes the result to <code>rd</code> .	0110011	000	0100000	<code>funct7[31:25]</code>
<code>sll rd, rs1, rs2</code>	Logical left shift of <code>rs1</code> by <code>rs2[4:0]</code> bits.	0110011	001	0000000	<code>funct7[31:25]</code>
<code>slt rd, rs1, rs2</code>	Writes 1 to <code>rd</code> if <code>rs1 < rs2</code> signed, else 0.	0110011	010	0000000	<code>funct7[31:25]</code>
<code>sltu rd, rs1, rs2</code>	Writes 1 to <code>rd</code> if <code>rs1 < rs2</code> unsigned, else 0.	0110011	011	0000000	<code>funct7[31:25]</code>
<code>xor rd, rs1, rs2</code>	Bitwise XOR between <code>rs1</code> and <code>rs2</code> .	0110011	100	0000000	<code>funct7[31:25]</code>
<code>srl rd, rs1, rs2</code>	Logical right shift of <code>rs1</code> by <code>rs2[4:0]</code> bits.	0110011	101	0000000	<code>funct7[31:25]</code>
<code>sra rd, rs1, rs2</code>	Arithmetic right shift of <code>rs1</code> by <code>rs2[4:0]</code> bits.	0110011	101	0100000	<code>funct7[31:25]</code>
<code>or rd, rs1, rs2</code>	Bitwise OR between <code>rs1</code> and <code>rs2</code> .	0110011	110	0000000	<code>funct7[31:25]</code>

Instruction	Description	opcode	funct3	funct7	Bitfield position
and rd, rs1, rs2	Bitwise AND between rs1 and rs2.	0110011	111	0000000	funct7[31:25]

Immediate arithmetic and logic (I-type)

Instruction	Description	opcode	funct3	funct7 / funct12	Bitfield position
addi rd, rs1, imm	Adds rs1 + imm sign-extended.	0010011	000	n/a	imm[31:20]
slti rd, rs1, imm	Writes 1 to rd if rs1 < imm signed, else 0.	0010011	010	n/a	imm[31:20]
sltiu rd, rs1, imm	Writes 1 to rd if rs1 < imm unsigned, else 0.	0010011	011	n/a	imm[31:20]
xori rd, rs1, imm	Bitwise XOR between rs1 and imm.	0010011	100	n/a	imm[31:20]
ori rd, rs1, imm	Bitwise OR between rs1 and imm.	0010011	110	n/a	imm[31:20]
andi rd, rs1, imm	Bitwise AND between rs1 and imm.	0010011	111	n/a	imm[31:20]
slli rd, rs1, shamt	Logical left shift of rs1 by shamt bits.	0010011	001	funct7=0000000	funct7[31:25]
srli rd, rs1, shamt	Logical right shift of rs1 by shamt bits.	0010011	101	funct7=0000000	funct7[31:25]
srai rd, rs1, shamt	Arithmetic right shift of rs1 by shamt bits.	0010011	101	funct7=0100000	funct7[31:25]

Upper immediate and PC-relative

Instruction	Description	opcode	funct3	funct7 / note	Bitfield position
lui rd, imm20	Loads imm20 into the high bits of rd (low bits zeroed).	0110111	n/a	U-type	imm20[31:12]
auipc rd, imm20	Writes the value PC + (imm20 << 12) to rd.	0010111	n/a	U-type	imm20[31:12]

Jumps and branches

Instruction	Description	opcode	funct3	funct7 / note	Bitfield position
jal rd, offset	PC-relative jump and return address saved in rd.	1101111	n/a	J-type	imm20[31:12]
jalr rd, rs1, imm	Indirect jump to rs1 + imm with return address in rd.	1100111	000	I-type	imm20[31:12]
beq rs1, rs2, offset	Branch if rs1 == rs2.	1100011	000	B-type	imm12[31:12]
bne rs1, rs2, offset	Branch if rs1 != rs2.	1100011	001	B-type	imm12[31:12]
blt rs1, rs2, offset	Branch if rs1 < rs2 signed.	1100011	100	B-type	imm12[31:12]
bge rs1, rs2, offset	Branch if rs1 >= rs2 signed.	1100011	101	B-type	imm12[31:12]
bltu rs1, rs2, offset	Branch if rs1 < rs2 unsigned.	1100011	110	B-type	imm12[31:12]
bgeu rs1, rs2, offset	Branch if rs1 >= rs2 unsigned.	1100011	111	B-type	imm12[31:12]

Memory accesses

Instruction	Description	opcode	funct3	funct7 / note	Bitfield position
lb rd, imm(rs1)	Loads 8 bits from memory with sign-extension.	0000011	000	I-type	imm[31:20], rs1[19:12]
lh rd, imm(rs1)	Loads 16 bits from memory with sign-extension.	0000011	001	I-type	imm[31:20], rs1[19:12]
lw rd, imm(rs1)	Loads 32 bits from memory.	0000011	010	I-type	imm[31:20], rs1[19:12]
lbu rd, imm(rs1)	Loads 8 bits from memory with zero-extension.	0000011	100	I-type	imm[31:20], rs1[19:12]
lhu rd, imm(rs1)	Loads 16 bits from memory with zero-extension.	0000011	101	I-type	imm[31:20], rs1[19:12]
sb rs2, imm(rs1)	Stores 8 bits (rs2[7:0]) to memory.	0100011	000	S-type	imm[11:5]=[31:25], rs2[7:0]
sh rs2, imm(rs1)	Stores 16 bits (rs2[15:0]) to memory.	0100011	001	S-type	imm[11:5]=[31:25], rs2[15:0]
sw rs2, imm(rs1)	Stores 32 bits of rs2 to memory.	0100011	010	S-type	imm[11:5]=[31:25], rs2[31:0]

System, CSR, and fence

Instruction	Description	opcode	funct3	funct7
<code>ecall</code>	Generates an environment call trap in M-mode.	1110011	000	000
<code>ebreak</code>	Generates a breakpoint trap in M-mode.	1110011	000	000
<code>csrrw rd, csr, rs1</code>	Writes <code>rs1</code> to CSR and returns the old value in <code>rd</code> .	1110011	001	CS1
<code>csrrs rd, csr, rs1</code>	Sets CSR bits with <code>rs1</code> , returns the old value in <code>rd</code> .	1110011	010	CS1
<code>csrrc rd, csr, rs1</code>	Clears CSR bits with <code>rs1</code> , returns the old value in <code>rd</code> .	1110011	011	CS1
<code>csrrwi rd, csr, zimm</code>	Writes <code>zimm</code> to CSR and returns the old value in <code>rd</code> .	1110011	101	CS1
<code>csrrsi rd, csr, zimm</code>	Sets CSR bits with <code>zimm</code> , returns the old value in <code>rd</code> .	1110011	110	CS1
<code>csrrci rd, csr, zimm</code>	Clears CSR bits with <code>zimm</code> , returns the old value in <code>rd</code> .	1110011	111	CS1
<code>fence</code>	Memory/ordering barrier (in v1 may be treated as a functional no-op).	0001111	000	pre
<code>fence.i</code>	Instruction fetch barrier (in v1 planned as not implemented).	0001111	001	imn

v1 implementation notes:

- Every instruction not implemented in the current RTL generates `illegal instruction`.
- Assembler pseudo-instructions (e.g., `li`, `mv`, `nop`, `j`, `ret`) are accepted by the toolchain but expanded into supported base RV32I instructions.

5.2 RISC-V instruction catalog (reference for ISA extensions)

This chapter lists the most common standard instructions to use as an architectural reference. For the v1 project we implement only the subset declared in 5.1.

RV32I (Base Integer ISA)

- `lui`, `auipc`
- `jal`, `jalr`
- `beq`, `bne`, `blt`, `bge`, `bltu`, `bgeu`
- `lb`, `lh`, `lw`, `lbu`, `lhu`
- `sb`, `sh`, `sw`
- `addi`, `slli`, `sltiu`, `xori`, `ori`, `andi`, `slli`, `srli`, `srai`
- `add`, `sub`, `sll`, `slt`, `sltu`, `xor`, `srl`, `sra`, `or`, `and`
- `fence`, `fence.i`
- `ecall`, `ebreak`
- `csrrw`, `csrrs`, `csrrc`, `csrrwi`, `csrrsi`, `csrrci`

RV64I/RV128I (not a v1 target, reference)

- `addiw`, `slliw`, `srliw`, `sraiw`
- `addw`, `subw`, `sllw`, `srlw`, `sraw`
- `ld`, `lduw`, `sd`

M Extension (Integer Multiply/Divide)

- `mul`, `mulh`, `mulhsu`, `mulhu`
- `div`, `divu`, `rem`, `remu`
- Additional RV64: `mulw`, `divw`, `divuw`, `remw`, `remuw`

A Extension (Atomic)

- lr.w, sc.w
- amoswap.w, amoadd.w, amoxor.w, amoand.w, amoor.w
- amomin.w, amomax.w, amominu.w, amomaxu.w
- Additional RV64: lr.d, sc.d, amoswap.d, amoadd.d, amoxor.d, amoand.d, amoor.d, amomin.d, amomax.d, amominu.d, amomaxu.d

F Extension (Single-Precision Floating Point)

- flw, fsw
- fmadd.s, fmsub.s, fnmsub.s, fnmadd.s
- fadd.s, fsub.s, fmul.s, fdiv.s, fsqrt.s
- fsgnj.s, fsgnjn.s, fsgnjx.s
- fmin.s, fmax.s
- fcvt.w.s, fcvt.wu.s
- fmv.x.w
- feq.s,flt.s, fle.s
- fclass.s
- fcvt.s.w, fcvt.s.wu
- fmv.w.x
- Additional RV64: fcvt.l.s, fcvt.lu.s, fcvt.s.l, fcvt.s.lu

D Extension (Double-Precision Floating Point)

- fld, fsd
- fmadd.d, fmsub.d, fnmsub.d, fnmadd.d
- fadd.d, fsub.d, fmul.d, fdiv.d, fsqrt.d
- fsgnj.d, fsgnjn.d, fsgnjx.d
- fmin.d, fmax.d
- fcvt.s.d, fcvt.d.s
- feq.d,flt.d, fle.d
- fclass.d
- fcvt.w.d, fcvt.wu.d, fcvt.d.w, fcvt.d.wu
- Additional RV64: fcvt.l.d, fcvt.lu.d, fcvt.d.l, fcvt.d.lu
- fmv.x.d, fmv.d.x (RV64)

C Extension (Compressed)

- c.addi4spn
- c.lw, c.sw
- c.nop, c.addi
- c.jal (RV32), c.addiw (RV64/128)
- c.li, c.addi16sp, c.lui
- c.srli, c.srai, c.andi
- c.sub, c.xor, c.or, c.and
- c.subw, c.addw (RV64/128)
- c.j, c.beqz, c.bnez
- c.slli
- c.lwsp
- c.jr, c.mv, c.ebreak, c.jalr, c.add
- c.swsp
- Additional RV64/128: c.ld, c.sd, c.ldsp, c.sdsp

Zicsr (CSR Instructions)

- csrrw, csrrs, csrrc, csrrwi, csrrsi, csrrci

Zifencei (Instruction-Fetch Fence)

- fence.i

Privileged instructions (Privileged ISA, depend on the supported mode)

- mret
- sret
- wfi
- sfence.vma
- hfence.vvma, hfence.gvma (hypervisor)

Notes:

- The set actually implemented by the core is the one declared in section 5.1.
- This catalog serves as a reference for future evolutions (RV32IM, RV32IMC, RV32IMAFD, etc.).
- In case of doubts about variants/edge cases, refer to the official RISC-V ISA specification.

6. Memory subsystem and address map (current RTL status)

6.1 Instruction cache

- `riscv_if_stage` instantiates `riscv_icache`; `riscv_imem` is not used in the current integrated datapath.
- Direct-mapped, read-only cache, with default parameters:
 - 128 lines.
 - 8 words of 32 bits per line (32 bytes).
 - total capacity 4 KiB.
- Each line has a tag and a valid bit; all valid bits are cleared on reset.
- Hit with combinational response; during a miss, the PC and the IF/ID register remain frozen until the refill completes.
- Full-line refill via a single 256-bit transaction on the I-side master.
- No software invalidation, no coherence with the D-cache, and no `fence.i` support.

6.2 Data cache

- `riscv_wb_stage` instantiates `riscv_dcache_ctrl`, which wraps `set_associative_cache`.
- Nominal default geometry:
 - 8 sets x 4 ways.
 - internal block of one 32-bit word.
 - total data capacity 128 bytes.
 - index `addr[4:2]`, tag `addr[31:5]`; way 3 used as the eviction candidate.
- Stores are write-back: each way carries a dirty bit, and a dirty eviction generates a write to RAM.
- `lb/lh/lbu/lhu` select byte/halfword and apply sign/zero extension in the controller.
- `sb/sh` use a read-modify-write sequence; `sw` writes the word directly.
- A load miss returns the word read directly from AHB but does not install it in the cache (`refill_i` is tied to zero). Repeated loads can therefore keep missing.
- Data accesses are multi-cycle. The FSM `ST_IDLE`, `ST_LOAD`, `ST_RMW_READ`, `ST_STORE`, `ST_DONE_A`, `ST_DONE_B` generates `mem_stall` to freeze IF and the ID/EX output registers.

D-cache limitations found in the current RTL:

- There are no valid bits in the ways. After reset the tags are zero, and an access with a zero tag can produce a false hit with zero data.
- In `ST_STORE`, `dc_en` and `dc_we` remain asserted for multiple cycles; the shift-chain structure of the ways can repeatedly insert the same entry and corrupt the associative state.
- Stall is asserted starting from the state following acceptance in `ST_IDLE`. ID/EX can advance on the first edge before the freeze takes effect, while the load destination and qualifier are not latched in the controller; load write-back is therefore not guaranteed end-to-end.
- No alignment checks are present. A halfword at offset 1 or 3 is selected based only on bit `addr[1]`; a word ignores `addr[1:0]`.
- The replacement policy declared in the module is not complete; the implementation uses a shift chain with way 3 as the eviction candidate.

6.3 Unified bring-up RAM

- `riscv_top` instantiates `riscv_ahb_ram_slave` as the shared backing store for the I-cache and D-cache.
- 32-bit byte address space; sparse storage indexed per 256-bit line.
- Uninitialized locations read as zero.
- Optional HEX file preload via `IMEM_HEX_FILE`, starting at `32'h0000_0000`, for `IMEM_WORDS` words (default 1024).
- Read and write have a registered address phase; the model keeps `HREADY=1` and `HRESP=OKAY`.
- A D-side write updates only the selected 32-bit lane within the line.
- The sparse associative array, the `exists` method, and the preload `initial` block make this a simulation model, not a synthesizable SRAM for ASIC.

6.4 Alternative/legacy memory modules

The following modules are present in the RTL folder but are not instantiated by `riscv_top`:

- `riscv_imem`, `riscv_dmem`: dedicated memories from the previous integration.
- `riscv_ahb_imem_slave`, `riscv_ahb_dmem_slave`: bus wrappers for the dedicated memories.
- `memory_wrapper`, `ram`, `cdc_sync_2ff`: alternative dual-clock cache/RAM path with CDC handshake.

These modules do not describe the current integrated datapath. They also have their own limitations: `riscv_dmem` effectively requires word-aligned addresses, `riscv_ahb_dmem_slave` keeps the internal request always active, and `ram` performs no range check before indexing.

7. Bus interface (current RTL status)

The `riscv` module exposes two reduced AHB-Lite master ports, absorbed internally by `riscv_top`:

- I-side: `HADDR`, `HTRANS`, `HWRITE=0`, `HSIZE`, `HRDATA`, `HREADY`, `HRESP`.
- D-side: same signals, with `HWRITE` and `HWDATA` for write-backs.
- Line-wide data payload, 256 bits with default parameters.
- Only `IDLE/NONSEQ` transfers, no bursts, `HPROT`, lock, or multiple outstanding transactions.
- Fixed D-side over I-side priority arbitration in the top. Continuous D-side traffic can starve fetch.
- `HRESP` reaches the caches but is not handled; no timeouts or traps for bus errors are present.

The interface is a custom subset compatible with the current internal integration, not a verified general AHB-Lite implementation. In particular, the RAM model samples `HWDATA` together with the address phase; internal masters and slaves use the same convention, but it does not represent the normal AHB-Lite address/data phase separation.

At the top level, `riscv_top` exposes only `clk_i`, `rst_ni`, and `illegal_instr_o`; the bus ports are internal to the wrapper.

8. Reset, clock, interrupt

- Single `clk_i` clock in the active integration.
- Asynchronous active-low reset used in pipeline, cache, and bus registers (`always_ff @(posedge clk_i or negedge rst_ni)`).
- Reset PC hardcoded to `32'h0000_0000`; no reset vector parameter exists.
- Interrupts not implemented.
- `memory_wrapper` contains a dual-clock path and 2-FF synchronizers, but it is not instantiated by the current top. The synchronizer is suitable for the single-bit handshake signals used in the wrapper, and does not guarantee atomicity for generic multi-bit buses.

9. CSR, traps, and errors

- CSRs are not implemented; there is no CSR file.
- `ecall`, `ebreak`, CSR, `fence`, and `fence.i` are decoded as illegal instructions.
- An unrecognized instruction asserts `illegal_instr_o` and inhibits write-back, but does not cause a redirect to a trap handler.
- No traps are implemented for misaligned accesses, access faults, instruction-address-misaligned, or bus errors.
- `jalr` forces bit 0 of the target to zero as required by RV32I; the resulting target's 32-bit alignment is not checked.

10. Hazard management and pipeline control

- Selective data forwarding:
 - EX -> EX from the registered ID/EX result, with priority over the older bypass.
 - WB -> EX from the final data written to the register file.
 - match on `rs1/rs2`, destination different from `x0`, and valid write enable.
- Load-use interlock present: inserts a bubble when `load_rd` matches `rs1` or `rs2` of the instruction in IF/ID.
- The load-use comparison does not qualify the actual use of the source fields based on the opcode and can produce unnecessary conservative stalls.
- There is no direct forwarding of load data from the D-cache to EX.
- `mem_stall` freezes PC/IF-ID and holds the ID/EX output registers during a multi-cycle D-cache access.
- The D-cache acceptance window described in 6.2 remains an open issue: the existing load-use interlock does not resolve the initial loss of load metadata.
- Branches and jumps are resolved in ID/EX without prediction. A taken redirect updates the PC and replaces the wrong-path instruction in IF/ID with a NOP, introducing a bubble.
- An I-cache miss autonomously keeps PC and IF/ID frozen until `rvalid` goes high again.

10.1 Functional compliance status

- ALU, immediates, branch/jump, and register file appear implemented consistently with the subset in section 5.1 based on static analysis.
- Load/store decode and byte/halfword handling are present, but the D-cache path requires fixes and regression testing before memory accesses can be declared compliant.
- Unimplemented instructions generate a flag, not an architectural trap.

11. Initial performance target (to be refined)

- Initial frequency target for the first synthesis runs: 200 MHz.

- Preliminary CPI target on simple code with cache hits: approximately 1.2 - 1.8.
- Taken branch: at least one bubble per flush, no prediction.
- Load/store: variable latency and pipeline stall; **sb/sh** add the read-modify-write sequence.
- I-cache misses and D-cache traffic share a single slave; D-side priority can increase fetch latency.
- Targets must be recalibrated after the D-cache fix and with Genus/Innovus reports on a synthesizable memory.

12. Explicit exclusions from v1

- MMU and virtual memory.
- Privilege state and modes beyond the nominal machine-level datapath.
- Trap handler, interrupts, and CSRs.
- ISA extensions A/F/D/C/M.
- Branch prediction, out-of-order execution, and multicore.
- I-cache/D-cache coherence and self-modifying code support.
- Complete debug hardware.
- Full AHB-Lite bus interface and synthesizable integrated RAM.

13. Minimum verification requirements

- Directed ISA tests for every instruction in section 5.1, including signed/unsigned and shift boundary values.
- Tests for EX -> EX and WB -> EX forwarding, forwarding priority, and x0 protection.
- Load-use tests with instructions using one or both operands, and cases that should not stall.
- Taken/not-taken branch/jump tests, forward/backward targets, wrong-path flush, and **jalr** bit 0.
- I-cache tests for cold miss, hit, conflict miss, refill, wait state, and valid bit reset.
- D-cache tests for cold access with zero tag, hit/miss on every way, dirty eviction, repeated loads, **sb/sh/sw**, sign extension, and read-modify-write.
- Assertions/checks on maintaining **load_valid** and **load_rd** from request acceptance through write-back.
- Simultaneous I-side/D-side starvation/arbitration tests and signal stability during wait states.
- Negative tests for illegal instruction, misalignment, and **HRESP**, documenting current behavior until traps are implemented.
- RTL code coverage and functional coverage for ISA classes, hazards, FSM states, hit/miss, eviction, and arbitration.

Note: the previous sequence based on dedicated IMEM/DMEM is not sufficient to validate the current cache/AHB top. End-to-end load/store compliance remains open until the issues in 6.2 are resolved.

14. Deliverables related to this specification

- IF, ID/EX, WB pipeline and ALU/register file modules.
- Direct-mapped I-cache with I-side master.
- D-cache controller and set-associative cache with write-back.
- I/D arbitration and unified simulation AHB RAM in the **riscv_top** wrapper.
- Legacy memory modules kept as alternative, inactive implementations.
- Updated testbench and regression for cache, bus, stall, and load/store.
- Initial clock/reset/IO constraints for synthesis.

15. RTL implementation status (complete snapshot)

15.1 Active datapath and control

- `riscv_top.sv`: wrapper, I/D arbiter, and unified RAM.
- `riscv.sv`: pipeline integration, forwarding, and interlock.
- `riscv_if_stage.sv`: PC, IF/ID, flush, and I-cache interface.
- `riscv_id_ex_stage.sv`: RV32I decode, immediates, ALU control, branch, and registers toward WB.
- `riscv_wb_stage.sv`: write-back mux and D-cache integration.
- `riscv_regfile.sv`: 32 x 32 bit, two asynchronous reads, one synchronous write.
- `riscv_alu_addsub.sv`, `riscv_alu_shift.sv`, `riscv_alu_logic.sv`, `riscv_alu_compare.sv`: dedicated combinational units.

15.2 Active memory subsystem

- `riscv_icache.sv`: direct-mapped I-cache and I-side refill.
- `riscv_dcache_ctrl.sv`: data access control, width/sign, RMW, and D-side master.
- `set_associative_cache.sv`: set selection, miss/write-back FSM, and RAM interface.
- `cache_set.sv`, `cache_way.sv`: four-way storage in a shift chain with dirty bit.
- `riscv_ahb_ram_slave.sv`: unified sparse simulation backing store.

15.3 Alternative modules not instantiated by the top

- `riscv_imem.sv`, `riscv_dmem.sv`.
- `riscv_ahb_imem_slave.sv`, `riscv_ahb_dmem_slave.sv`.
- `memory_wrapper.sv`, `ram.sv`, `cdc_sync_2ff.sv`.

15.4 Parametrization and synthesis constraints

- `LINE_LENHT` is defined as `2**LINE_WORDS`; this relationship coincides with 256 bits only for the default `LINE_WORDS=8`, but the general correct width would be `LINE_WORDS*32`.
- `ICACHE_LINE_WORDS` and `LINE_WORDS` are separate parameters and must be kept consistent manually.
- `set_associative_cache` extracts index and tag with fixed slices for 8 sets and 32-bit blocks; the declared parametrization is not fully general.
- `cache_set` uses an `assign #1ns` delay, unsuitable for a portable synthesizable datapath.
- `riscv_ahb_ram_slave` is a sparse behavioral model and must be replaced with synthesizable RAM/ROM in the ASIC flow.

16. Open issues and priorities for the v1 freeze

1. Fix the ID/EX-D-cache contract: assert stall in the acceptance cycle, or latch all metadata, including `load_valid` and `load_rd`.
2. Add valid bits to the D-cache and make way updates single pulses; verify replacement and dirty eviction.
3. Define and implement the load refill policy, including possible read allocation.
4. Define the behavior for misaligned accesses and add a trap or explicit block.
5. Fix the line parametrization (`LINE_WORDS*32`) and unify the I-cache/system bus parameters.
6. Make the interface AHB-Lite compliant, or rename/document it as a custom internal protocol; add `HRESP` handling.
7. Replace sparse RAM and RTL delays with synthesizable implementations and define the ROM/SRAM/peripheral map.
8. Add cache/bus/load-store regression before declaring the RV32I subset fully compliant.

9. Decide on including the M extension, interrupt/CSR/trap strategy, and **fence.i** support.
10. Decide whether to remove legacy modules from the compile list or keep them in a separate test configuration.